

NeuroSpector: Systematic Optimization of Dataflow Scheduling in DNN Accelerators

Chanho Park^{ID}, Bogil Kim^{ID}, Sungmin Ryu^{ID}, and William J. Song^{ID}, *Member, IEEE*

Abstract— This paper presents an optimization framework named *NeuroSpector* that systematically analyzes the dataflow of deep neural network (DNN) accelerators and rapidly identifies optimal execution methods. The proposed methodology is demonstrated to work effectively with a variety of accelerator architectures and DNN workloads. It has been a baffling challenge to devise scheduling schemes for neural accelerators to maximize energy efficiency and performance. The challenge lies in that hardware specifications associated with multi-dimensional DNN data create an enormous number of possible scheduling options that can be exerted on accelerators. Related work suggested various techniques to solve the challenge encompassing brute-force search of massive solution spaces pruned by user constraints, solving the objective functions of system models, learning-based optimization, etc. However, each suggested technique was devised only for a specific accelerator model. Therefore, we find that they are not adaptively applicable to different accelerators and DNN workloads in that they produce hit-or-miss results with 100.1% greater energy and cycles on average compared to optimal scheduling schemes obtained from fully comprehensive brute-force searches. In contrast, *NeuroSpector* identifies efficient execution methods for various accelerators and workloads with only 1.5% differences on average to the optimal scheduling solutions. The optimization strategy of *NeuroSpector* is based on an observation that optimal executions are strongly correlated with minimizing data movements to the lower-level memory hierarchy of accelerators rather than maximizing the utilization of processing elements. Thus, *NeuroSpector* prioritizes optimizing lower-level components in the accelerator hierarchy, which is proven highly effective for various accelerators and DNN workloads.

Index Terms—Deep neural network, accelerator, dataflow, mapping, scheduling, optimization.

I. INTRODUCTION

A NUMBER of hardware accelerators have been proposed to speed up deep neural network (DNN) computations and enhance energy efficiency [4], [5], [6], [11], [19], [20], [26], [34]. DNN accelerators offer high-throughput and energy-efficient computing solutions by deploying many processing elements

(PEs) and chips in parallel and exploiting data reuse across multiple levels of the memory hierarchy in the accelerators. The vertical and spatial arrangements of buffers and PEs construct a multi-level *hierarchy* of accelerator components from multiply-accumulate (MAC) units to global buffers and off-chip DRAM. For diverse DNN workload configurations and accelerator implementations, it is a highly challenging problem to find the proper ways of executing neural layers on accelerators to maximize energy efficiency and performance.

Scheduling a DNN workload in a neural accelerator needs to determine in which order multi-dimensional data have to be processed and how they are partitioned to fit into on-chip buffers and spatially laid out onto many PEs. We refer to the first issue that deals with the computation order as *dataflow* and the second problem of data partitioning and distribution as *mapping* in this paper. The challenge lies in that hardware specifications (e.g., number of PEs and chips, buffer sizes and types) associated with workload configurations (e.g., width, height, channel, batch size) produce a huge number of possible dataflow and mapping options that can be exercised in an accelerator. In addition, optimal dataflow and mapping solutions vary drastically across different workloads and hardware designs. The scheduling problem becomes more complicated as recent accelerators advocate reconfigurable architectures [6], [10], [14], [26] that can adaptively support variable dataflow and mapping schemes for different DNNs.

Several approaches were proposed to tackle the challenge of dataflow and mapping optimization in DNN accelerators. Suggested techniques in related work encompass brute-force search of massive solution spaces curtailed by user constraints [7], [30], [41], solving the objective functions of system models [17], learning-based optimization [3], [13], [21], heuristic search across accelerator hierarchy [2], [28], etc. However, each technique was devised only for a specific accelerator architecture and tested with a small set of DNN workloads. Thus, we observe in our experiments that the prior techniques [3], [7], [13], [17], [28], [30] cannot effectively handle different types of accelerators and workloads, and even occasionally encounter algorithmic dead ends. On average, they exhibit 2.0x greater energy and cycles than the optimal scheduling schemes obtained from fully comprehensive brute-force searches and as much as 14.9x in the worst case. Since the prior techniques simply rely on generally occurring routines without detailed investigations into their working mechanisms, they cannot effectively handle different accelerators and DNN workloads.

Manuscript received 1 August 2022; revised 30 May 2023; accepted 2 June 2023. Date of publication 7 June 2023; date of current version 15 June 2023. This work was supported by the National Research Foundation (NRF) of Korea under Grant #2021R1A2C1095162. Recommended for acceptance by A.C. Zhou. (Chanho Park and Bogil Kim contributed equally to this work.) (Corresponding author: William J. Song.)

The authors are with the School of Electrical and Electronic Engineering, Yonsei University, Seoul 03722, South Korea (e-mail: ch.park@yonsei.ac.kr; bogilkim@yonsei.ac.kr; sungmin.ryu@yonsei.ac.kr; wjhsong@yonsei.ac.kr).

Digital Object Identifier 10.1109/TPDS.2023.3283491

To overcome the limitations of previous efforts, this paper introduces *NeuroSpector* that systematically analyzes the dataflow and mapping possibilities of DNN workloads in accelerators and rapidly identifies optimal scheduling schemes. The optimization strategy of *NeuroSpector* is based on an observation that the optimal executions are strongly correlated with the energy and cycle costs of off-chip DRAM. It may sound obvious since off-chip memory accesses are costly, so it is important to minimize them. However, the obvious statement was not properly exploited in the prior studies. They were generally biased toward finding answers that yield high PE utilization, which we find is not as strongly correlated with optimal executions. Based on this observation, *NeuroSpector* evaluates dataflow and mapping solutions by traversing the accelerator hierarchy in a bottom-up manner such that the resulting scheduling schemes minimize costly accesses to lower-level buffers and off-chip DRAM. This strategy is demonstrated to be most effective for identifying optimal scheduling solutions for many different accelerators [4], [6], [11], [19], [20], [34] and neural networks [8], [9], [15], [16], [22], [31], [33], [35], [36], [38]. It also helps reduce the size of problem spaces to evaluate by gradually and rapidly diminishing the number of possible scheduling options as the optimization flow steps upward in the accelerator hierarchy. As a result, *NeuroSpector* can identify a dataflow and mapping solution from a massive solution space with only 1.5% energy or cycle difference on average to the optimal scheme within a second of calculation time, whereas the previous techniques incur as much as 7958x greater algorithm complexity only to produce a highly deviant answer. Importantly, the systematic analysis of *NeuroSpector* can clearly explain why a certain scheduling scheme has to be used for a given DNN workload in an accelerator.

In addition, the prior methods [3], [7], [13], [17], [28], [30], [41] do not support many important features of accelerator architectures. The suggested techniques can handle only narrow scopes of DNN workloads and accelerators, but they still struggle with finding optimal scheduling solutions in a timely manner. In contrast, *NeuroSpector* offers a robust optimization methodology while supporting various accelerator features such as multi-chips, network-level optimization, group convolutions, etc. The following highlights the key technical contributions of *NeuroSpector*.

- *Accurate, rapid, and robust scheduling optimization:* *NeuroSpector* outperforms prior techniques by accurately and rapidly identifying optimal scheduling solutions for various neural networks and accelerators.
- *Versatile modeling capability of accelerator features:* *NeuroSpector* is capable of handling various workload and accelerator features such as multi-chips, group convolutions, buffer types (e.g., separate, shared), data bypassing in buffers, and consideration for static power, which were missing or only partly supported in related work.
- *Network-wise, multi-chip scheduling optimization:* Most previous methods disregarded correlations between neural layers and focused only on per-layer scheduling solutions. *NeuroSpector* supports network-wise scheduling via

cross-layer optimization and multi-chip partitioning for parallelizable neural layers.

II. BACKGROUND

This section overviews how DNN layers and accelerators are formed, and we present their generalized representations that can be extended to express diverse neural layers and accelerator designs. DNN dataflow and mapping cases in neural accelerators are described based on these representations.

A. DNN Layer Representation

Fig. 1(a) shows the formation of a neural layer. It consists of three *data types*; input, weight, and output. The layer data are comprised of eight dimensional *parameters* including batch size (B), input channel (C), group size (G), weight width (R), weight height (S), weight count (K), output width (P), and output height (Q). Input width and height (i.e., U and V) can be derived from other parameters, so they do not have unique notions.

Noticeably, every parameter, except for G , is related only to two data types. Fig. 1(b) summarizes the correlations between data types and parameters. For instance, the input channel, C , is related to both input and weight but not to output. These correlations shed light on data reuse cases in neural computations.

Processing multi-dimensional DNN data creates deeply nested loops as shown in Fig. 1(c). Importantly, a different loop order has a completely different pattern and amount of data reuse. The example in Fig. 1(c) shows that weight-related parameters (i.e., G , K , C , R , S) are placed in outer loops, and inner loops have weight-irrelevant parameters (i.e., B , P , Q). Thus, weight data are reused in the inner loops, and this example shows a case of weight-stationary (WS) dataflow. Other cases of output-stationary (OS) and input-stationary (IS) dataflows can be similarly composed by reordering the loops [4]. As such, the pattern and amount of data reuse can be manipulated by reordering the loops. However, they become significantly complicated when the computation is laid out in an accelerator and constrained by hardware limitations (e.g., buffer sizes, number of PEs and chips).

The data representation in Fig. 1 is extensible to expressing many different neural layers, including fully connected layers, depth-wise convolutions, shape transformation, tree-based aggregation, etc. Since the key of accelerator scheduling is to reduce data transfers between off-chip memory, on-chip buffers, and execution units to minimize the number of operations and thus the resulting energy and latency costs, operation types (e.g., MAC, MAX, MIN) are of less concern. Thus, the scheduling optimization problem is formulated as finding the solution of the most efficient data movements, and the data expression in Fig. 1 can be easily applied to various other DNN operations with minor changes in data dimensions from the scheduling optimization perspective. For instance, shape transformation and reshaping can be expressed by adjusting output-related parameters and setting the weight size to zero. Similarly, tree-based aggregation can be represented by fixing the height and width of weight data to zeros.

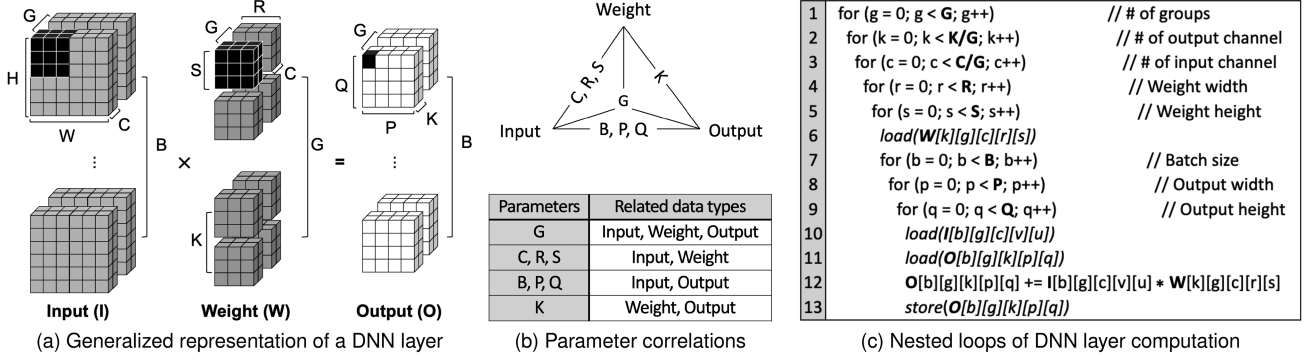


Fig. 1. (a) A DNN layer consists of three data types (i.e., input, weight, output) that are defined by eight parameters (i.e., B, G, K, P, Q, C, R, S); U and V of the input can be derived from other parameters. This representation method can express many different forms of neural layers. (b) Each parameter, except for G , is associated only with two data types. For example, K is related to weight and output but not input. (c) Computing multi-dimensional DNN data creates deeply nested loops. A loop order determines the pattern and amount of data reuse in the computation.

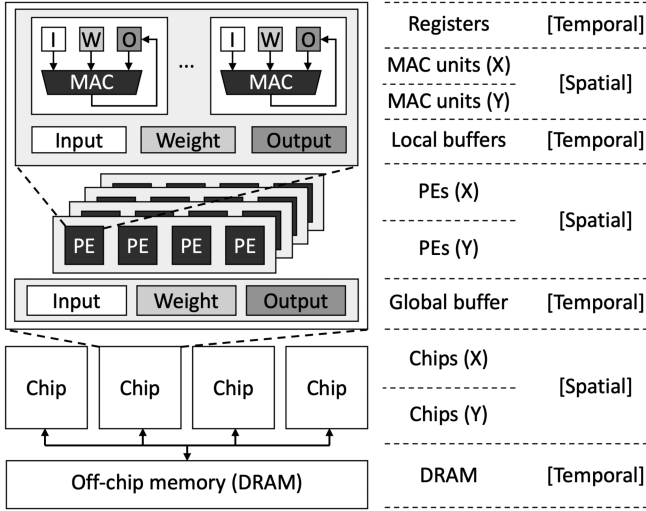


Fig. 2. A DNN accelerator has multiple levels of component hierarchy from MAC registers to off-chip memory. Accelerator components at different levels exploit spatial or temporal reuse of DNN data. This representation of the accelerator hierarchy can be easily extended to express a variety of accelerator designs.

B. DNN Accelerator Representation

Every accelerator has seemingly different configurations and features [5], [6], [11], [20], [34], but they share common architectural constructs that can be expressed in a generalized form as shown in Fig. 2. An accelerator is organized around spatially and vertically arranged components forming a multi-level hierarchy from MAC registers to off-chip memory.

To process a DNN layer, the off-chip memory contains a set of input, weight, and output data. A part of the data set is loaded to the *global buffer* of a chip-level accelerator for processing. We refer to a subset of data loaded to a buffer at a time as a *data tile*. Data tiling occurs not only from the DRAM to the global buffer but also between on-chip buffers along the memory hierarchy of the accelerator. The size of a data tile determines the extent of data reuse opportunity at each level of the memory hierarchy, and thus it has a significant impact on accelerator efficiency. In the case of a multi-chip design [6], [34], different parts of data are

loaded to the global buffer of discrete chip-level accelerators for parallelism. A global buffer holds large data tiles to be accessed and reused by *processing elements*. The global buffer plays a crucial role in determining the performance and efficiency of the accelerator by supplying data to PEs and reducing costly off-chip memory accesses. The accelerator employs a large number of PEs to exploit data-level parallelism. The PEs are typically arranged in a 2-D grid with X and Y-directional placements. Each PE includes a set of buffers to store input, weight, and output data tiles. The data buffers in a PE are called *local buffers*. A set of data buffers can be fused into a unified structure (e.g., scratchpad) instead of having separate buffers for different data types. The local buffers store small tiles of input, weight, and output data for MAC units. A PE in many cases has only one MAC unit, but it can have an array of them for greater throughput [11], [34].

Importantly, accelerator components at different hierarchy levels exhibit different data reuse behaviors. On-chip buffers such as local and global buffers exploit temporal reuse of data, and spatial arrangements of MAC units, PEs, and chips exploit spatial data reuse. The accelerator can be modeled as a stack of *spatial* and *temporal*-level components along the hierarchy as shown in Fig. 2. The on-chip buffers enable the temporal reuse of data by retaining data tiles, so they are the places where different sorts of stationary cases are formed (i.e., IS, WS, and OS dataflows); a stationary dataflow occurs where data can stay. The spatial arrangements of components help increase data-level parallelism and computational throughput. Collectively, there can be a huge number of possible scheduling options for processing a neural layer, considering dataflow options at multiple levels of the memory hierarchy and mapping possibilities across spatially and vertically arranged components in the accelerator. Although the concepts of spatial and temporal data reuse were introduced in the prior works [13], [28], [30], they only dealt with them from a loop ordering perspective. NeuroSpector extends the concepts to model accelerator architecture as the hierarchical organization of spatial and temporal-level components.

Fig. 3 plots the energy and cycle variations of all possible scheduling options to execute a YOLO v3 layer [31] on Eyeriss v2 [6]. There are about 13 million possible cases of executing the layer in the accelerator with drastically different energy and

TABLE I
COMPARISON OF DNN ACCELERATOR SCHEDULING OPTIMIZATION METHODS [3], [7], [13], [17], [28], [30], [41]

Optimization methods		Unorganized brute-force search			Obj. functions	Learning-based		Hierarchical search	
Schedulers		Timeloop	Interstellar	dMazeRunner	CoSA	Mind Mappings	TVM	ZigZag	NeuroSpector
Modeling capabilities	Group conv.						✓	✓	✓
	Multi-chips	✓						✓	✓
	Bypassing	✓			✓	✓		✓	✓
	Buffer types	✓			✓			✓	✓
	Static power								✓
Network-wise optimization									✓
Optimization accuracy		Random	Hit or miss	Hit or miss	Highly deviant	Highly deviant	Highly deviant	Hit or miss	Near-optimal
Runtime per layer		up to 25 min	up to 20 min	up to 10 min	1-5 sec	5-30 sec	up to 10 min	up to 5 min	Less than a sec

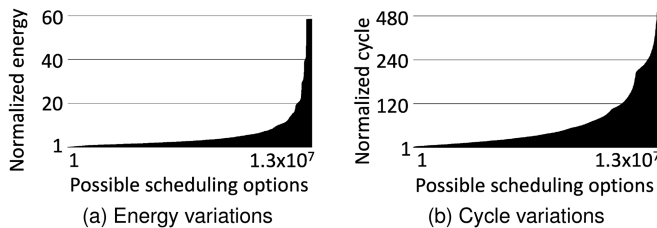


Fig. 3. Energy and cycle variations for different scheduling options in an accelerator. There are about 13 million possible scheduling solutions to execute one YOLO v3 layer [31] on Eyeriss v2 [6]. Selecting different dataflow and mapping options results in drastically different energy and cycle consequences.

cycle outcomes. The scheduling options are sorted in ascending order based on their energy or cycle results normalized to the optimal scheme (i.e., minimum energy or cycle). We used the energy and cycle models of Accelergy [39] and CACTI [27] in the experiment. The graphs highlight the significance and difficulty of finding the optimal scheduling scheme in the massive problem space. In this paper, we focus on the scheduling problem of dense DNNs and not on exploiting data sparsity in accelerators since value-dependent data sparsity is hard to predict offline and correctly account for scheduling optimization.

III. RELATED WORK

To tackle the scheduling optimization challenge of DNN accelerators, several efforts have been made [2], [3], [7], [13], [17], [28], [29], [30], [37], [41]. Related work can be categorized into four major approaches; i) unorganized brute-force search of huge solution spaces curbed by user constraints, ii) mathematically solving the objective functions of system models, iii) learning-based optimization, and iv) hierarchical search of accelerator organizations. Table I compares the speed, optimization accuracy, and modeling capabilities of several milestone works. The listed ones in the table have their codes publicly available, so we could investigate the details of their optimization algorithms.

Timeloop [30] presented energy and cycle cost models based on Accelergy [39] to evaluate accelerator efficiency. Other subsequent related works [13], [17], [28] adopted the cost models of Timeloop, and so does NeuroSpector. However, Timeloop uses a random search method to find a scheduling solution. It keeps searching scheduling options until it encounters a case that beats other randomly generated schemes more than a certain number of times. If the search limit is set to infinite, it becomes a complete brute-force search. Its search accuracy is natively

random and strongly depends on the search limit. For finding a scheduling solution of a single DNN layer, it spends as much as 25 minutes depending on the problem complexity using its default search limit, but its random search still ends up with a highly erroneous answer. In addition, it does not support modeling group convolutions, static power, and network-wise optimization. Evaluating only the dynamic energy of accelerator operations often leads to irrational solutions such as activating only a few PEs with extremely long execution cycles. Also, Timeloop performs only per-layer optimization and misses potential correlations between DNN layers. Such limitations prohibit its use from layer-to-layer optimization or multi-chip partitioning of parallelizable layers. Other related works also have the same inherent problems.

Interstellar [41] and dMazeRunner [7] also leverage unorganized brute-force search but restrict search spaces based on the utilization thresholds of PEs, local and global buffers. The utilization thresholds make a trade-off between search time and optimization accuracy, but they require users to provide the constraints explicitly. They can be effective for finding optimal scheduling solutions when the utilization numbers exhibit good correlations with the energy and cycle results of an accelerator. However, they often produce highly deviant answers from the optimal or even give up on finding one, which indicates that simply maximizing the utilization of PEs and buffers does not guarantee the discovery of the optimal solutions. In addition, they have very limited modeling capabilities in that they cannot handle group convolutions, multi-chip accelerators, bypassing (i.e., skipping data in buffers if no reuse is expected), different buffer types (i.e., separate data buffers versus unified data buffers), static power, and network-wise optimization.

CoSA [17] defines a few objective functions for modeling an accelerator by considering buffer utilization, execution cycles, communication traffic, etc. Then, it solves the objective functions to derive a scheduling option. However, there is no mathematical proof that the objective functions represent the accelerator system, and there is no clear way of refining the objective functions for different accelerators. The optimization accuracy of CoSA was mistakenly compared to the random search results of Timeloop in the paper [17]. When we validated it against the complete brute-force search, it showed highly erroneous results.

Mind Mappings [13] leverages a neural network to estimate the energy and cycle costs of a scheduling option in an accelerator. The neural network is used for refining the scheduling option toward minimizing the costs. The updated scheduling

option is fed back to the neural network to re-evaluate energy and cycle costs. The feedback loop repeats a number of times set by a user, and the resulting scheduling option at the end of the iterations is taken as a final answer. The accuracy of this approach depends on the coverage of a training set, but we find it mostly inaccurate in our experiments. Similarly, TVM [3] leverages a learning algorithm to find scheduling solutions. It trains its learning-based model to infer the costs of scheduling options. Based on the cost predictions, TVM tunes the hardware mapping of scheduling options using a simulated annealing algorithm [23], which is essentially a random search method. Its optimization accuracy depends on the maturity of the training set and the number of tuning steps, but we also find it highly inaccurate in our experiments.

ZigZag [28] performs a level-by-level search along an accelerator hierarchy from PEs to off-chip memory in a top-down manner. Such an approach intrinsically favors scheduling options that yield high PE utilization. Similar to Interstellar and dMazeRunner, leveraging the PE utilization as a decision criterion frequently leads to deviant answers from the optimal scheduling schemes or even fails to find one (i.e., no valid scheduling solutions satisfying the utilization constraints).

Marvel [2] focuses on minimizing data transfers between an accelerator and DRAM. However, its accelerator model comprises only two levels (i.e., on-chip processor and off-chip memory) and disregards the internals of the accelerator architecture. Since optimal scheduling schemes are governed by complex correlations between spatially and vertically arranged accelerator components, minimizing only the off-chip memory traffic does not necessarily lead to optimal scheduling solutions. In addition, an unorganized brute-force search requires exhaustive evaluations of possible scheduling candidates, and thus it is subject to excessive search complexity. As far as we know, Marvel does not have its code publicly available, so it is precluded from further consideration in this paper and not shown in Table I.

The common problem of previous works [2], [3], [7], [13], [17], [28], [29], [30], [37], [41] is that each technique was tested only for one specific accelerator and a narrow set of DNN workloads. Hence, we observe in our experiments that they fail to find optimal scheduling schemes or even reach algorithmic dead ends when an accelerator model is modified, even with a small change such as the number of PEs or buffer size. It implies that every accelerator needs a customized method for scheduling optimization, which is impractical and unsustainable. This paper takes on this challenge and proposes a robust scheduling optimization methodology that is demonstrated to work effectively with a variety of accelerators and neural networks. In addition, the prior techniques do not support many important accelerator features as summarized in Table I. Although NeuroSpector adopted energy and cycle cost models from Timeloop [30] similar to other related works, we have upgraded the models to add the missing features, including group convolutions, static power, layer-to-layer and tile-to-tile data reuse in on-chip buffers, and multi-chip accelerators.

Correlations	W/O	O/I				I/W			I/W/O	Data-flow
Parameters	K	B	P	Q	C	R	S	G		
Register	K_{Reg}	B_{Reg}	P_{Reg}	Q_{Reg}	C_{Reg}	R_{Reg}	S_{Reg}	G_{Reg}		IS
MAC (X)	$K_{MAC:X}$	$B_{MAC:X}$	$P_{MAC:X}$	$Q_{MAC:X}$	$C_{MAC:X}$	$R_{MAC:X}$	$S_{MAC:X}$	$G_{MAC:X}$		-
MAC (Y)	$K_{MAC:Y}$	$B_{MAC:Y}$	$P_{MAC:Y}$	$Q_{MAC:Y}$	$C_{MAC:Y}$	$R_{MAC:Y}$	$S_{MAC:Y}$	$G_{MAC:Y}$		-
Local buffer	K_{LB}	B_{LB}	P_{LB}	Q_{LB}	C_{LB}	R_{LB}	S_{LB}	G_{LB}		WS
PE (X)	$K_{PE:X}$	$B_{PE:X}$	$P_{PE:X}$	$Q_{PE:X}$	$C_{PE:X}$	$R_{PE:X}$	$S_{PE:X}$	$G_{PE:X}$		-
PE (Y)	$K_{PE:Y}$	$B_{PE:Y}$	$P_{PE:Y}$	$Q_{PE:Y}$	$C_{PE:Y}$	$R_{PE:Y}$	$S_{PE:Y}$	$G_{PE:Y}$		-
Global buffer	K_{GB}	B_{GB}	P_{GB}	Q_{GB}	C_{GB}	R_{GB}	S_{GB}	G_{GB}		OS
Chip (X)	$K_{Chip:X}$	$B_{Chip:X}$	$P_{Chip:X}$	$Q_{Chip:X}$	$C_{Chip:X}$	$R_{Chip:X}$	$S_{Chip:X}$	$G_{Chip:X}$		-
Chip (Y)	$K_{Chip:Y}$	$B_{Chip:Y}$	$P_{Chip:Y}$	$Q_{Chip:Y}$	$C_{Chip:Y}$	$R_{Chip:Y}$	$S_{Chip:Y}$	$G_{Chip:Y}$		-
DRAM	K_{DRAM}	B_{DRAM}	P_{DRAM}	Q_{DRAM}	C_{DRAM}	R_{DRAM}	S_{DRAM}	G_{DRAM}		-

Fig. 4. The scheduling table displays the dataflow and mapping layout in an accelerator. It lists the hierarchical level of accelerator components in rows and DNN layer parameters in columns. The last column specifies which dataflows are exercised by on-chip buffers. Each element in the scheduling table indicates the mapping value of a layer parameter at the given component level.

IV. METHODOLOGY

This section describes the methodology of NeuroSpector for the scheduling optimization of DNN accelerators. The optimization methodology is formulated as solving dataflow and mapping problems based on the representation of neural layers and accelerators explained in Section II.

A. Dataflow and Mapping Representations

Based on DNN parameter correlations described in Fig. 1(b) and the generalized representation of the accelerator hierarchy in Fig. 2, NeuroSpector formulates a *scheduling table* as shown in Fig. 4. The scheduling table arranges DNN parameters in the accelerator hierarchy. It lists the hierarchical level of accelerator components in rows and the layer parameters in columns. The DNN parameters are enumerated based on their correlations with input, weight, and output data types. The last column indicates which stationary dataflows are associated with temporal-level components (i.e., buffers). Each element in the table is the *mapping value* of a DNN parameter at the given component level. For instance, P_{LB} means the amount of P -dimensional data to be stored in a local buffer. Similarly, $K_{PE:X}$ indicates the spatial distribution degree of K -dimensional data across PEs in the X direction.

To help the understanding of the scheduling table, Fig. 5 shows a simplified example. An accelerator is assumed to have only three levels in the hierarchy; local buffers, two PEs, and DRAM. A neural layer also simply consists of four parameters, $R = 3$, $S = 3$, $P = 2$, and $Q = 2$. All other DNN parameters are assumed to be 1 and not explicitly shown. Filling in the scheduling table must satisfy both parameter and hardware constraints. In particular, multiplying all the numbers in the same column must match the parameter value of the neural layer. In the case of P , the product of all numbers in the corresponding column (i.e., $P_{LB} \times P_{PE:X} \times P_{DRAM}$) must be equal to $P = 2$. Other parameters must also comply with the constraint rule. Numbers

Correlations	O/I		I/W	
Parameters	P	Q	R	S
Local buffer	1	1	3	1
PE (X)	2	1	1	1
DRAM	1	2	1	3

Hardware constraints:
 tile size $\leq$ buffer size
 • Input tile size ($U_{LB} \times V_{LB}$) = 3
 • Weight tile ($R_{LB} \times S_{LB}$) = 3
 • Output tile ($P_{LB} \times Q_{LB}$) = 1

Parameter constraint:
 $P = P_{LB} \times P_{PE \cdot X} \times P_{DRAM} = 2$

Fig. 5. A simplified example of the scheduling table shows that an accelerator has only three levels in the component hierarchy (i.e., local buffers, two PEs, DRAM), and a neural layer consists of only four parameters (i.e., R , S , P , Q). Filling in the scheduling table must satisfy both hardware and parameter constraints.

TABLE II
FORMULAS FOR CALCULATING DATA TILE SIZES AND ACCESS COUNTS

Metrics	Data types	Formulas
Data tile size	Input	$\prod B_i \cdot C_i \cdot U_i \cdot V_i$
	Weight	$\prod (K_i/G_i) \cdot C_i \cdot R_i \cdot S_i$
	Output	$\prod B_i \cdot K_i \cdot P_i \cdot Q_i$
Tile access count	Baseline	$\prod B_i \cdot (K_i/G_i) \cdot P_i \cdot Q_i \cdot C_i \cdot R_i \cdot S_i$
	Input	Baseline/ (K_b/G_b) if IS Otherwise, baseline
	Weight	Baseline/ $(B_b \cdot P_b \cdot Q_b)$ if WS Otherwise, baseline
	Output	Baseline/ $((C_b/G_b) \cdot R_b \cdot S_b)$ if OS Otherwise, baseline

in the same row are subject to hardware constraints. If a row represents a temporal level (e.g., local buffer), the tile size of input, weight, and output data must be smaller than or equal to respective buffer sizes. In the case of a unified buffer, the sum of data tile sizes must meet the capacity limit. The tile size of each data type can be calculated using formulas in Table II. For instance, the size of a weight tile is calculated as $R_{LB} \times S_{LB}$ since other parameters of the weight tile formula (i.e., K , G , C) are 1 in the example. Similarly, if a row corresponds to a spatial level, the product of all numbers in the row represents the number of active components participating in the layer computation. The product value must be less than or equal to the total number of components in the spatial direction to meet the hardware constraint (e.g., two PEs in the X direction). Although a dataflow column is omitted in the scheduling table in Fig. 5, the local buffer row should specify which dataflow is engaged with it. The amount of data transfers in and out of the buffer differs depending on the dataflow, which affects the energy and cycle costs of the accelerator. As described, the scheduling table displays how DNN layer parameters are laid out in the accelerator.

Table II shows formulas to calculate the size of data tiles and their access counts at a temporal level (i.e., buffer row) in the scheduling table. The data tile size associated with the tile-granular access count tells the total amount of data transferred to a buffer. The formulas are adopted from Timeloop [30] but rearranged according to the hierarchical organization of accelerator components in the scheduling table because Timeloop and other related works [7], [25], [28], [41], [42] formulated cost calculations in the loop ordering aspect. In NeuroSpector, data tile sizes are used for checking compliance with buffer capacity

limits and evaluating the energy and cycle costs of scheduling options during an optimization process. Using Table II, the size of a data tile to be stored in a buffer is calculated as the product of mapping values in a formula. For instance, the size of an output data tile is calculated by multiplying four output-related parameters, B , K , P , and Q . Since each row of the scheduling table only has mapping values at the given component level, the product of DNN parameters is multiplied from the buffer level upward to the top of the scheduling table to find the total tile size. For example, the size of an output data tile to be stored in a global buffer is calculated by multiplying the product of $B \cdot K \cdot P \cdot Q$ from the global buffer to register rows in the scheduling table in that data buffers are inclusive.

The bottom half of Table II shows formulas for calculating tile access counts. Data tiles are repeatedly loaded into a buffer from the lower-level memory or buffer in a loop. Thus, the tile-granular access count of the buffer is equivalent to the number of loop iterations. It is calculated as the product of DNN parameters using the baseline formula in Table II. For instance, the total number of weight tiles loaded into local buffers across all PEs is calculated by multiplying the product of DNN parameters in the baseline formula from one level below the local buffer row to the bottom of the scheduling table, which is from the PE (X) level to the DRAM row. Importantly, the tile access count differs depending on which dataflow is utilized by the buffers. If the local buffers are engaged with WS dataflow, the tile access count of the baseline formula is divided by weight-irrelevant parameters (i.e., $B_b \cdot P_b \cdot Q_b$ for b = global buffer) because weight tiles are reused while input and output data tiles are streaming. In the case of output tiles, they do not need initial loads into the buffers but require writing partial sums or final outputs back to the global buffer. Such variations are all considered in the energy and cycle calculations of NeuroSpector. Since the amount and pattern of data transfers are deterministically identifiable for given layer configurations, dataflow types, and accelerator specifications, the energy and cycle costs of different scheduling options can be instantly calculated and compared.

Each stationary dataflow can be composed in multiple ways by reordering DNN parameter loops. Changing a loop order alters the transient sequence of data transfers, but it does not affect the total number of tile-granular data movements. Thus, it only needs to differentiate dataflow types, and cost formulas in Table II do not differ regarding loop orders. For instance, Fig. 1(c) shows a loop sequence of WS dataflow in that weight-relevant parameters repeat in outer loops. Once a weight tile is loaded at line 6 in the example, the data tile is reused while looping through weight-irrelevant parameters of B , P , and Q in inner loops. The inner loops can be freely reordered without changing the total number of tile access counts, and similarly the outer loops above line 6 can be rearranged as well. An exception happens when a specified dataflow type has no technical effects. For example, the WS dataflow in Fig. 1(c) becomes ineffective if the DNN parameters of inner loops are all one. In this case, a weight data tile loaded at line 6 is never reused because nothing technically repeats in the inner loops. This loop sequence effectively turns into IS or OS dataflow depending on the order of the outer loops, whose tile access count should then be calculated using the IS

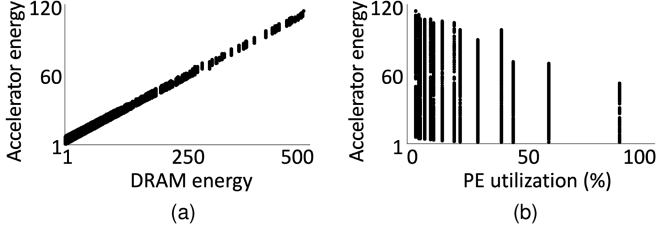


Fig. 6. Correlations of accelerator energy with (a) the amount of energy spent on DRAM accesses and (b) PE utilization (i.e., the ratio of active PEs) for different scheduling options to execute a YOLO v3 layer [31] on Eyeriss [5]. The DRAM energy cost is more closely related to the accelerator energy than the PE utilization, and thus it gives a better hint on finding the optimal scheduling scheme.

or OS formula in Table II. When such an exception occurs, NeuroSpector evaluates both IS and OS dataflows instead of the ineffectual WS dataflow.

B. Scheduling Optimization Methodology

The optimization methodology of NeuroSpector works on the scheduling table to find a dataflow and mapping combination that gives the minimum energy or cycle cost. It uses two evaluation strategies to identify the optimal execution scheme of a neural layer in a DNN accelerator.

1) *Minimizing the Costs of Lower-Level Components:* The main optimization strategy of NeuroSpector is, in short, a bottom-up approach that walks along the scheduling table from the bottom rows upward. It is devised based on an observation that optimal scheduling schemes are strongly correlated with the energy and cycle costs of the off-chip DRAM. In contrast, prior techniques [7], [28], [29], [41] prioritize scheduling options that give high PE utilization in an assumption that increasing the PE utilization would naturally decrease costly accesses to lower-level buffers and the off-chip memory.

Fig. 6 plots how the total energy of an accelerator is correlated with the DRAM energy cost and PE utilization. Every data point in the graphs is a possible scheduling option to execute a YOLO v3 layer [31] on Eyeriss [5] as an exemplary neural layer and accelerator using the energy models of Accelergy [39] and CACTI [27]. The energy numbers are normalized to the optimal scheme (i.e., minimum energy). Fig. 6(a) shows that the total accelerator energy is strongly correlated with the amount of energy spent on DRAM accesses since decreasing the DRAM energy cost approaches the minimum accelerator energy. On the other hand, Fig. 6(b) reveals that maximizing the PE utilization does not necessarily lead to the minimum accelerator energy. In particular, at the max PE utilization of 86% in the graph, there are as wide as 51.5x energy variations among the possible scheduling options. Since the limited on-chip buffers of the accelerator cannot fully accommodate sizable DNN data, the layer data has to be tiled and repeatedly transferred from the off-chip memory, which has dominant impacts on energy and cycle costs. Thus, DRAM costs show a strong correlation with the overall accelerator-level energy and cycle results, whereas the PE utilization has a relatively weaker correlation. This explains

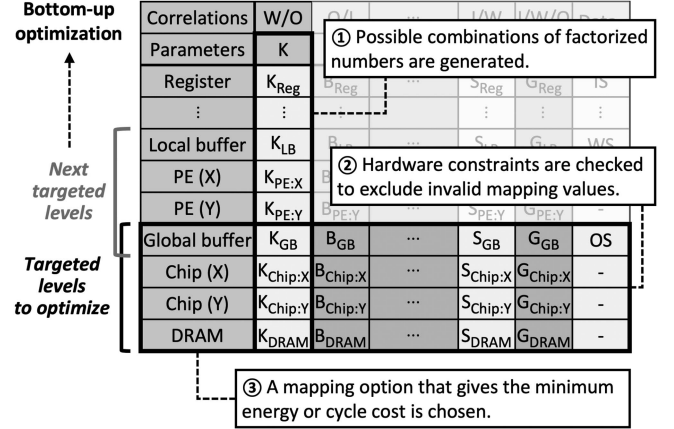


Fig. 7. The optimization flow of NeuroSpector fills in the scheduling table from the bottom rows. NeuroSpector works on a set of rows at each optimization step from one temporal level to another. Possible combinations of factorized numbers are generated, and their validity is checked against hardware constraints. Then, a combination of mapping values that gives the minimum energy or cycle cost is chosen. This process repeats for the next target levels upward in the scheduling table.

why chasing after the PE utilization in the previous works [7], [28], [29], [41] often selects deviant scheduling options from the optimal schemes. Cycle variations also exhibit similar trends as Fig. 6 in that reducing the cycle cost of the DRAM is likely to approach the minimum execution time, but increasing the PE utilization is not necessarily so.

To exploit this observation, NeuroSpector fills in the scheduling table from the bottom rows. This approach gives optimization priorities to lower-level components in the accelerator hierarchy such that a derived scheduling scheme minimizes costly accesses to lower-level buffers and the off-chip memory. At each optimization step, NeuroSpector works on a set of rows in the scheduling table from one temporal level to another, as illustrated in Fig. 7. The accelerator hierarchy has four temporal levels; DRAM, global buffer, local buffer, and register as specified in Fig. 2. Hence, the first target levels to optimize are from the DRAM to global buffer rows, the next target rows are from the global buffer to the local buffer, and so on. At each optimization step working on a set of levels, what NeuroSpector tries to find is i) data tile sizes that fit into buffers, ii) spatial distribution degrees of DNN parameters across chips, PEs, or MAC units, whichever are the target levels, and iii) finally mapping values that give the minimum energy or cycle cost at the target levels in the scheduling table.

The optimization flow of NeuroSpector shown in Fig. 7 takes the following steps. ① For the first four levels to optimize at the bottom of the scheduling table, NeuroSpector generates all possible combinations of factorized numbers for DNN parameters. The factorized numbers are placed at the bottom three levels; DRAM, chip (Y), and chip (X). The upper levels of the scheduling table are yet to be optimized, so the products of upper-level mapping values are temporarily recorded at the fourth level, which in this case is the global buffer row. Regardless of how factorized numbers are distributed in the upper rows, their collective products do not change. For instance,

suppose a DNN layer has $K = 64$. If a scheduling option has $K_{\text{DRAM}} = 4$, $K_{\text{Chip:Y}} = 1$, and $K_{\text{Chip:X}} = 1$, multiplying all the upper-level numbers must be 16 to match $K = 64$. The number 16 is temporarily recorded at the place of K_{GB} and will be factorized later at the next optimization step if this scheduling option is chosen as predictably the best one. ② Hardware constraints are checked at the target levels except for the bottom-most row, which in this case are chip (Y), chip (X), and global buffer rows. Hardware constraints confirm if the size of data tiles calculated using the formulas in Table II fits into the global buffer and if the factorized numbers of DNN parameters satisfy the number of chips in the X and Y directions. Scheduling options (i.e., combinations of mapping values) failing to satisfy the hardware constraints are precluded from further consideration. ③ The remaining scheduling options are all valid ones. NeuroSpector quickly evaluates their energy or cycle costs at the target levels and picks the one that yields the minimum cost. The steps of ①–③ repeat for the next target levels from the global buffer to local buffer rows, and again from the local buffer to register rows. This bottom-up optimization flow gradually and rapidly diminishes the number of scheduling options to evaluate as it steps upward in the scheduling table.

The described flow is the primary optimization method of NeuroSpector. It gives optimization priorities to lower-level components in the accelerator hierarchy such that it minimizes costly accesses to them, based on the observation that optimal scheduling schemes are strongly correlated with the cost metrics of the lower-level components. In our experiments, this strategy successfully finds optimal scheduling schemes for 83% of the test cases.

2) *Balancing Upper and Lower-Level Components*: Optimal scheduling schemes tend to have large mapping values at the upper rows of the scheduling table to increase the size of data tiles loaded into on-chip buffers and the number of active PEs and MAC units involved in computations. Improving the utilization of upper-level components (e.g., local buffers and PEs) in the accelerator hierarchy and reducing the energy and cycle costs of lower-level components (e.g., global buffer and DRAM) are related to each other, but they are rather loosely correlated than directly coupled. In other words, maximizing the utilization of upper-level components does not necessarily guarantee to minimize costly accesses to lower-level components and vice versa because accelerator energy and cycle costs are governed by complex relations between dataflow and mapping in the accelerator hierarchy as manifested in the formulas of Table II.

NeuroSpector takes the bottom-up optimization method as explained in Section IV-B1 by giving optimization priorities to lower-level components in the accelerator hierarchy. Optimizing the bottom levels first in the scheduling table may result in finding a sub-optimal solution at the upper rows, but spatially and vertically arranged upper-level components still have plenty of possible scheduling options that exhibit similar energy and cycle costs as the optimal scheduling scheme. On the other hand, taking a top-down approach results in choosing a sub-optimal solution at the bottom rows of the scheduling table, but lower-level components in the accelerator hierarchy

do not have as diverse scheduling options to explore while they have substantial impacts on the overall energy and cycle costs. Therefore, top-down optimization methods toward increasing the PE utilization [7], [28], [29], [41] often end up with deviant scheduling options or even fail to find one for reaching an algorithmic dead end. Instead of taking level-by-level optimization approaches, a few other related studies attempted to solve accelerator scheduling problems in more holistic ways, such as using learning-based optimization or solving the objective functions of system models [3], [13], [17], [21], [37], but we find them highly erroneous in our experiments.

Scheduling optimization based on energy and cycle cost evaluations along the accelerator hierarchy in a bottom-up manner successfully finds optimal execution schemes in most cases. However, making decisions using the cost metrics occasionally leaves large prime numbers at the later steps of the optimization flow, which are unfavorable to ensure the diversity of possible scheduling options to explore. A prime number is not factorizable and can result in the underutilization of upper-level components in the accelerator hierarchy because the prime number has to be placed in the scheduling table in an all-or-nothing fashion. Having large prime numbers at the later steps of the optimization flow significantly constrains the cases of explorable scheduling options, which may lead to picking up a non-optimal answer from the limited selection pool.

To discourage the cases of carrying large prime numbers to the upper rows of the scheduling table, NeuroSpector leverages another strategy that deals with the factorization of DNN parameters. This supplementary strategy is not a separate procedure from the main optimization flow but differs only in choosing a scheduling option. At each optimization step working on a set of rows in the scheduling table, NeuroSpector selects two scheduling candidates, i) one that minimizes the energy or cycle cost of target levels as described in Section IV-B1 and ii) another that offers the largest factorization degree (i.e., the number of factorized values) for upper-level components in the accelerator hierarchy. These two candidates are concurrently explored and passed to the next optimization step. When the optimization procedure reaches the top of the scheduling table, NeuroSpector finally picks the scheduling option with the lowest energy or cycle cost. The supplementary strategy ensures that upper-level components in the accelerator hierarchy can have sufficient diversity of scheduling options to explore.

Fig. 8 illustrates how NeuroSpector evaluates the factorization degree of a DNN parameter. ① For the first four levels to optimize at the bottom of the scheduling table, possible combinations of factorized numbers are generated for each DNN parameter; the main optimization flow of NeuroSpector already does it. ② Among the combinations that satisfy hardware constraints, NeuroSpector inspects the factorization degree of the values placed at the upper temporal level (i.e., global buffer row in the example). Since the numbers will be factorized later in the next step of the optimization flow, their factorization degrees indicate scheduling flexibility at the upper rows of the scheduling table. For instance, 38 can only be divided into 19 and 2 (i.e., factorization degree of 2), but 32 can be factorized into five twos. NeuroSpector selects 32 rather than

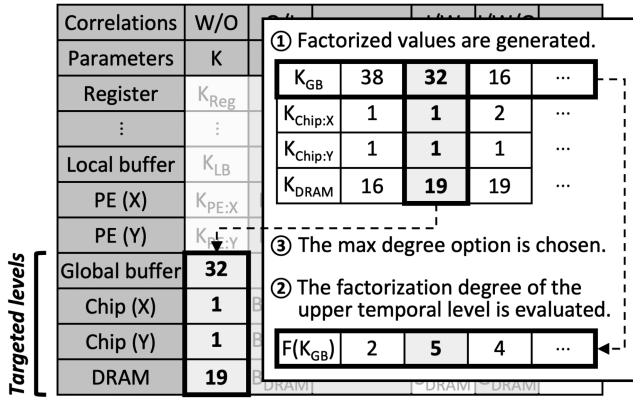


Fig. 8. At each step of the optimization flow, NeuroSpector also finds a scheduling option that has the highest factorization degree of DNN parameters. It ensures that upper-level components have sufficient diversity of scheduling options to explore by avoiding large prime numbers. This scheduling option is passed to the next optimization step together with the scheduling scheme that has the minimum energy or cycle cost.

38, although this option leaves a greater amount of data in the DRAM (i.e., $K_{DRAM} = 19$ over 16) and increases the number of memory accesses. However, avoiding large prime numbers at the upper rows of the scheduling table can potentially better utilize resources and subdue the increased memory access penalty. As such, NeuroSpector takes two scheduling candidates at each step of the optimization procedure, one that minimizes the energy or cycle cost of target levels and another that offers the largest factorization degree for upper rows. In our experiments, this strategy finds optimal scheduling schemes for 17% of the test cases.

C. Network-Level Optimization

Most related studies regarding the scheduling optimization of DNN accelerators [2], [7], [13], [17], [28], [29], [30], [37], [41] treated individual neural layers independently and focused only on deriving per-layer scheduling solutions. Since neural layers in a network are related to each other, there exist opportunities to leverage the relations to further enhance the accelerator performance and efficiency. Although a few previous works [1], [40] considered data sharing between neighboring neural layers, they were limited to the simplest case where DNN data were fully loaded onto the large on-chip buffer of single-chip processors. In contrast, NeuroSpector supports scheduling optimization for more complex cases where DNN data do not entirely fit into the on-chip buffers of multi-chip accelerators. NeuroSpector implements network-wise scheduling via i) cross-layer optimization and ii) multi-chip partitioning.

1) *Cross-Layer Optimization*: Layers in a deep neural network are interconnected with each other such that the output of a layer becomes the input of the next layer. A part or even all of the input data does not have to be loaded from the off-chip memory but can be directly reused in the global buffer of an accelerator. Cross-layer optimization requires comparing the size of output and input tiles between adjacent layers, which falls into one of three cases shown in Fig. 9(a).

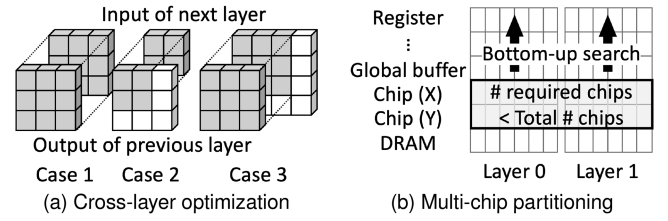


Fig. 9. (a) NeuroSpector supports cross-layer scheduling optimization by considering data reuse between neural layers. (b) Scheduling optimization for concurrent layers across multi-chips can be achieved in a similar way as the bottom-up optimization of individual layers. It only needs to check if hardware constraints (i.e., # of chips) are met when multiple layers are put together for all combinatorial cases.

In case 1, a pair of output and input data tiles are equal in size. Nothing is written back to or loaded from the off-chip memory. In case 2, the size of an output tile is greater than that of an input tile of the next layer. The next layer can skip the initial load of the input data, but the non-reused portion of the output tile (i.e., unshaded edges) has to be written back to the memory. Case 3 is the opposite of case 2 in that an output tile is smaller than an input tile of the next layer. The output data are not written back, but the uncovered portion of the input has to be loaded from the DRAM. NeuroSpector considers these layer-to-layer data reuse cases when calculating the energy or cycle costs of scheduling options during the optimization flow. Related experimental results are discussed in Section V-F.

2) *Multi-Chip Partitioning*: In some DNNs such as Inception [35], [36], multiple layers branch off from the same root layer and then merge back. The branch layers do not have dependencies on each other, so they can be executed concurrently to exploit chip-level parallelism if such executions help improve the performance. The bottom-up optimization method of NeuroSpector effectively identifies the optimal way of multi-chip partitioning for the concurrent executions of independent layers.

Fig. 9(b) presents the conceptual illustration of scheduling optimization for concurrent multi-chip executions. The example assumes that two layers can be executed simultaneously across multiple chips. NeuroSpector first generates possible scheduling options (i.e., mapping values) at the bottom levels of the scheduling table for each neural layer independently. The mapping values are used for examining if they satisfy hardware constraints (i.e., number of chips in the X and Y directions) when multiple layers are put together. Possible cases in this example are i) executing each layer one at a time and ii) executing both layers together. If there are more than two parallelizable layers, there are simply more combinatorial cases to examine. Checking the hardware constraints will screen out most combinations and leave only a few to evaluate their energy or cycle costs. Among the possible scheduling options, NeuroSpector takes two scheduling candidates, one with the minimal cost and another with the largest factorization degree. Once the number of chips assigned to each neural layer is identified, the rest of the optimization procedure is identical to per-layer scheduling and can proceed independently. More discussions with experimental results will follow in Section V-F.

TABLE III
DNN ACCELERATOR CONFIGURATIONS

Accelerators	Eyeriss v1	Eyeriss v2	TPU v3	Simba	SparTen
MAC per PE	1	2	1	64	32
Local buffer (I/W/O)	24/448/48 bytes	24/288/80 bytes	0/1/0 bytes	8/32/3 kilobytes	4/8/2 kilobytes
PEs (X×Y)	14×12	4×3	256×128	4×4	32×1
Global buffer	108KB	12KB	32MB	64KB	N/A
Dataflow (LB/GB)	WS/OS	WS/OS	WS/OS	WS or OS/WS or OS	WS/-
# of chips	1	2×8	2×4	6×6	1

TABLE IV
NEURAL NETWORK CHARACTERISTICS

Networks	# Layers	Data size	Features
YOLO v3	75	221MB	Large data, non-pow-2 filters
Inception v4	149	83MB	Asymmetric weights
MobileNet v3	57	8.2MB	Depth-wise, point-wise conv.
CvT	121	29.0MB	Convolutional transformer
BERT	76	202MB	Transformer for NLP
GCN	2	14MB	Graph neural network
ResNet-50	50	38MB	Bottleneck layers

V. EVALUATION

A. Experiment Setup and Validation

The scheduling optimization efficacy of NeuroSpector is evaluated with various DNN accelerators and workloads. The tested accelerators include Eyeriss [5], [6], TPU [18], Simba [34], and SparTen [11] as specified in Table III. Eyeriss has an ordinary form of accelerator architecture, and the successive v2 model features a multi-chip configuration. TPU boasts sizable global buffers compared to other accelerators, and Simba utilizes a large number of MAC units per PE. Both have multi-chip designs. SparTen is originally for exploiting data sparsity, but in this paper we borrow only its architectural construct and assume the processing of dense DNNs. Notably, SparTen does not have global buffers. In conjunction with the hardware accelerators, we use YOLO v3 [32], Inception v4 [35], MobileNet v3 [15], CvT [38], BERT [8], GCN [22], and ResNet-50 [12] in our experiments as listed in Table IV.

NeuroSpector calculates the energy and cycle costs of accelerator scheduling schemes using the formulas in Table II, based on the energy and latency models of Accelerger [39] and CACTI [27]. The cost models of NeuroSpector were validated against Timeloop [30] for fair comparative experiments. Other related works [13], [17], [28] also used the Timeloop models. Fig. 10 compares the breakdown of dynamic energy expenses between NeuroSpector and Timeloop for selected neural layers on Simba [34] when they use the same scheduling schemes as a snapshot of validation efforts. We preformed rigorous validation of NeuroSpector models with more DNN workloads [9], [16], [24], [31] on various accelerators [5], [6], [11], [18], [34]. As distinctive features, NeuroSpector supports static power modeling in cost calculations, cross-layer optimization, and multi-chip partitioning, which were overlooked in previous studies.

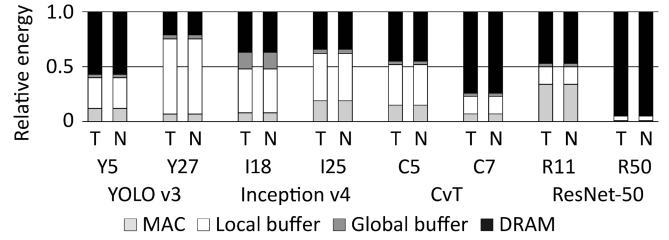


Fig. 10. Comparison of dynamic energy breakdowns between Timeloop (T) and NeuroSpector (N) for selected neural layers on Simba [34], including YOLO v3, Inception v4, CvT, and ResNet-50 when they use the same scheduling schemes. Labels such as Y5 and Y27 denote layer positions in a neural network. Cost models of NeuroSpector were validated against Timeloop [30] for fair comparative experiments since many other related works [13], [17], [28] also used the Timeloop models.

B. Accuracy of Scheduling Optimization

Scheduling optimization for DNN accelerators needs to solve dataflow and mapping problems. The challenge lies in that accelerator specifications associated with workload configurations produce an enormous number of possible dataflow and mapping options that can be exercised in the accelerators. The bottom-up optimization flow of NeuroSpector using a scheduling table effectively resolves the challenge. The efficacy of NeuroSpector is demonstrated with a variety of neural accelerators and networks in Tables III and IV, and it is compared to prior techniques, including Timeloop [30], dMazeRunner [7], CoSA [17], ZigZag [28], Mind Mappings [13], and TVM [3]. In particular, dMazeRunner, Mind Mappings, and TVM have very limited modeling capabilities in that they are unable to handle any accelerators listed in Table III. Hence, they were upgraded to support different buffer types and data bypassing features at least, which are supported in all other tools. The others are tested with their default models and optimization algorithms in the experiments.

Fig. 11 compares relative accelerator energy based on scheduling schemes derived from different tools for various DNNs on Eyeriss and single-chip Simba. Every neural layer requires a different scheduling method for the most energy-efficient execution. The optimal scheduling scheme of each DNN layer is obtained from a fully comprehensive brute-force search, and it is compared to the outcomes of different tools to evaluate how close their answers are. Gray bars in the graphs show relative accelerator energy aggregated at the network level, and error ranges (i.e., black lines with bar ends or numbers) are peak-to-peak variations of layer-wise results relative to their optimal scheduling schemes.

Fig. 11(a) shows that Timeloop (T), dMazeRunner (D), CoSA (C), ZigZag (Z), Mind Mappings (M), and TVM (V) consume 1.79x, 1.05x, 1.58x, 1.09x, 2.94x, and 2.99x greater energy on average than the optimal scheduling schemes, respectively. However, the average numbers are only for the layers that the tools could find scheduling solutions. As shown in the graphs, there are many cases where the prior techniques fail to obtain any scheduling options. The cases plotted as $\boxtimes$ in the graphs (e.g., T for MobileNet) indicate that the tools do not support the

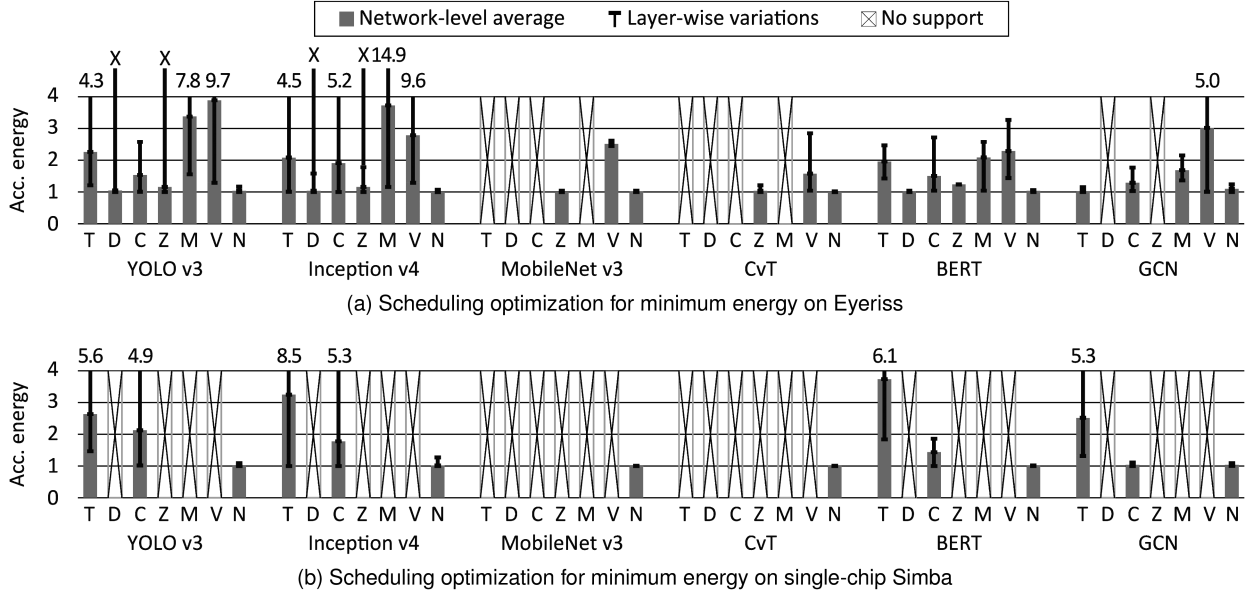


Fig. 11. Relative accelerator energy based on scheduling solutions derived from Timeloop (T), dMazeRunner (D), CoSA (C), ZigZag (Z), Mind Mappings (M), TVM (V), and NeuroSpector (N) for various neural networks on (a) Eyeriss and (b) Simba. Gray bars show accelerator energy aggregated at the neural network level, and lines with bar ends indicate layer-wise variations and the worst-case layer results. Cases shown as $\boxtimes$ (e.g., T for MobileNet) indicate that they cannot handle the given networks or accelerators. $\times$ marks above the error bars (e.g., D for YOLO) imply that they give up on finding scheduling solutions for several layers in the networks.

networks or accelerators because of lacking essential modeling features. For instance, Timeloop, dMazeRunner, CoSA, and Mind Mappings cannot handle MobileNet and CvT for missing the support of group convolutions. The prior techniques also often give up on finding scheduling solutions by encountering algorithmic dead ends, even though they have necessary modeling supports. These cases demonstrate the incompetence of the optimization algorithms. For example, dMazeRunner fails to find any scheduling options for 18.8% of neural layers in Inception, and ZigZag also gives up on finding solutions for 8.3% of layers in the same network. To denote such cases, their peak errors are shown as $\times$ in the graphs. Removing all the failed tests leaves only a few working sets, such as CoSA with Inception. However, the results show that executing Inception layers in the way that CoSA tells will consume 1.9x more energy on average than optimal scheduling and as much as 5.2x for some layers. Other tools also exhibit similar problems in that derived scheduling schemes are far from optimal and have large variations across different layers in the neural networks.

Fig. 11(b) shows another set of experiments for the same DNNs but on single-chip Simba. Most prior techniques fail the tests for various reasons, as the graph is mostly filled with $\boxtimes$ cases. Simba features 64 MAC units per PE as specified in Table III, but dMazeRunner, ZigZag, Mind Mappings, and TVM cannot handle the accelerator that incorporates an array of MAC units in PEs. Timeloop and CoSA generate highly inefficient scheduling schemes for YOLO and Inception, and they do not support the group convolutions of MobileNet and CvT. If the experiments were made with multi-chip Simba, none of the prior techniques would process the scheduling optimization of

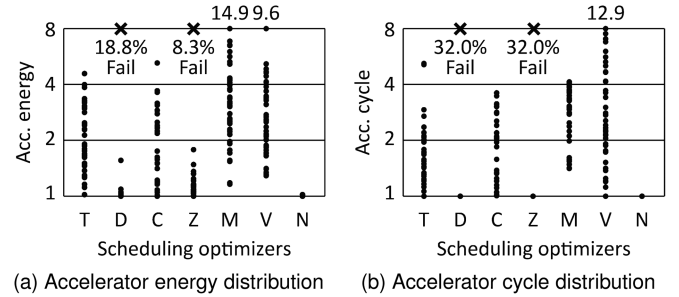


Fig. 12. Distributions of normalized layer-wise results derived from different scheduling optimization tools for processing Inception v4 layers toward (a) minimizing accelerator energy on Eyeriss and (b) minimum execution cycles on SparTen.

the multi-chip accelerator but Timeloop only with erroneous outcomes.

Fig. 12(a) plots the distribution of layer-wise energy results toward minimizing accelerator energy when processing Inception v4 layers on Eyeriss. The graph shows that the prior techniques suffer from wide variations of optimization accuracy across different layers. Timeloop (T), CoSA (C), Mind Mappings (M), and TVM (V) struggle with scheduling optimization in general with large deviance. dMazeRunner (D) and ZigZag (Z) show hit-or-miss behaviors, and notably they fail to derive any scheduling options for 18.8% and 8.3% of layers in the network, respectively. In contrast, NeuroSpector accurately finds optimal scheduling solutions with small variations across all layers. Similarly, Fig. 12(b) shows the distribution of layer-wise results toward minimizing execution cycles on SparTen with Inception

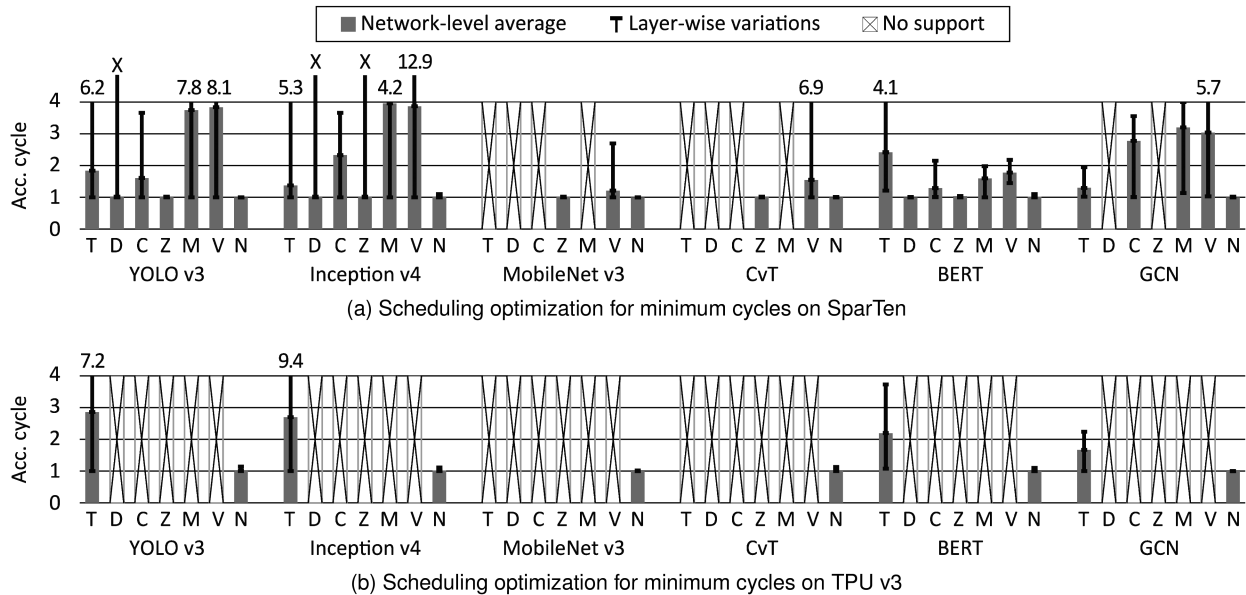


Fig. 13. Relative accelerator cycles based on scheduling schemes derived from different tools toward minimum-cycle executions on (a) SparTen and (b) TPU v3. The prior techniques cannot handle many test cases (e.g., M for CvT), and their optimization algorithms frequently give up on finding solutions due to algorithmic dead ends (e.g., Z for Inception with an X mark above the error bar). Gray bars are network-level averages, and lines with bar ends show layer-wise variations. Cases shown as X (e.g., C for MobileNet) indicate that they do not support such cases.

layers. The overall trends are similar to Fig. 12(a). Timeloop, CoSA, Mind Mappings, and TVM generally have deviant results with wide variations across layers. dMazeRunner and ZigZag show hit-or-miss behaviors, giving up on finding scheduling options for 32.0% of layers in the network. On the other hand, NeuroSpector outperforms the prior techniques by accurately identifying optimal scheduling solutions with small variations across layers without algorithmic dead ends.

Fig. 13 plots the results of scheduling optimization toward minimum-cycle executions for the same DNNs as Fig. 11 but on SparTen and TPU v3 to show the effectiveness of NeuroSpector for different neural accelerators and optimization targets. In this case, optimizations are made to find scheduling schemes with minimum execution cycles, so every test case selects a different scheduling option from what was used in Fig. 11. For the simple architecture of SparTen without global buffers, ZigZag works relatively well by favoring scheduling schemes that yield high PE utilization because missing the global buffers makes spatial data reuse across PEs play as the primary mechanism for the scheduling optimization. However, ZigZag still fails to deal with many Inception layers on SparTen, giving up on finding scheduling solutions for them. Other tools show highly deviant results from optimal scheduling as manifested in Fig. 13(a). Since most prior tools do not support multi-chip designs, there are not many valid data in Fig. 13(b) but NeuroSpector and Timeloop. However, Timeloop struggles with scheduling optimization for minimum-cycle executions on TPU.

In all cases, NeuroSpector outperforms the prior optimization techniques by delivering scheduling solutions with only 1.5% energy and cycle differences on average to the optimal executions for a variety of DNN accelerators and neural networks. Noticeably, the optimization results of NeuroSpector exhibit small

variations across layers in the neural networks, which implies the robustness and efficacy of its optimization methodology.

C. Comparison of Algorithm Complexity

The bottom-up optimization methodology of NeuroSpector rapidly narrows down the number of scheduling options to evaluate as the optimization flow steps upward in the scheduling table. Fig. 14 shows the relative complexity of scheduling optimization algorithms for processing DNN workloads on Eyeriss. The algorithm complexity is measured as the number of incurred operations instead of the number of assessed scheduling options since the optimization algorithms of related work differ in how they evaluate the scheduling options. On average, Timeloop, dMazeRunner, CoSA, ZigZag, Mind Mappings, and TVM show 5067x, 320x, 84x, 479x, 245x, and 495x greater algorithm complexity over NeuroSpector needless to mention the inaccuracy of their optimization results.

The hybrid search method of Timeloop requires many iterations to gain confidence in random search outcomes, and it is the slowest among the tested tools. dMazeRunner and ZigZag rely on the utilization thresholds of PEs and on-chip buffers to constrain search scopes, but there still exist a large number of possible scheduling options to explore. Notably, restricting the search spaces with the utilization constraints frequently gives up on finding any scheduling solutions because there are no scheduling schemes satisfying the constraints. CoSA uses the objective functions of system metrics to derive a scheduling option, but solving the objective functions incurs a non-trivial number of iterative calculations as manifested in the algorithm complexity. Mind Mappings and TVM generally evaluate fewer scheduling options than other techniques, but their

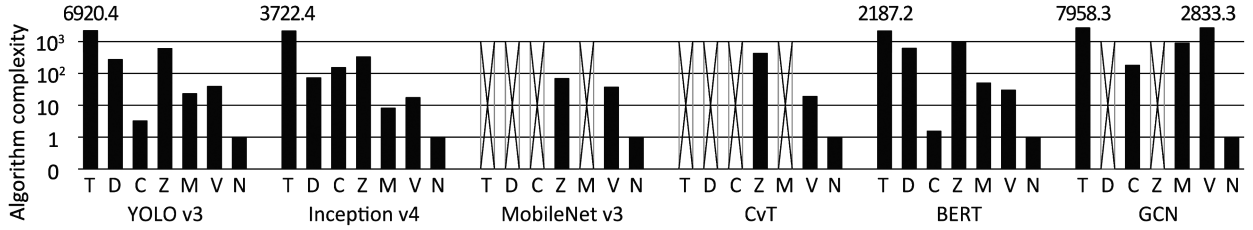


Fig. 14. Relative complexity of optimization algorithms for processing neural networks on Eyeriss. NeuroSpector has substantially lower algorithm complexity than Timeloop (T), dMazeRunner (D), CoSA (C), ZigZag (Z), Mind Mappings (M), and TVM (V). Cases shown as $\times$ are invalid tests since they do not support such neural networks.

learning-based methods are inherently heavy to execute. In comparison, NeuroSpector outperforms all the related works in both optimization accuracy and algorithm simplicity.

D. Bottom-Up Optimization and DRAM Costs

The primary optimization strategy of NeuroSpector gives optimization priorities to lower-level components in the accelerator hierarchy based on the observation that optimal scheduling schemes are strongly correlated with the access costs of the off-chip memory. Fig. 15 reveals the efficacy of the bottom-up optimization flow for minimizing costly memory accesses and deriving optimal scheduling solutions. Fig. 15(a) plots normalized accelerator energy for executing two layers of YOLO v3 (i.e., Y4, Y14) and another two layers of Inception v4 (i.e., I8, I16) on Eyeriss using the scheduling schemes obtained from different tools. Labels such as Y4 and Y14 denote the layer positions in a neural network. Fig. 15(b) shows the respective energy spent on DRAM accesses.

The two graphs of Fig. 15(a) and (b) reveal close correlations between the total accelerator energy and the DRAM energy cost. When a scheduling option shows large deviance from optimal such as CoSA (C) for Y14 or ZigZag (Z) for I8, it is due to having excessive memory accesses and thus a large DRAM energy cost. On the contrary, PE utilization ratios in Fig. 15(c) do not exhibit such strong correlations with the total accelerator energy. dMazeRunner, CoSA, and ZigZag favor the scheduling options of high PE utilization, but the derived solutions are not necessarily energy-efficient. As shown, DRAM costs provide a better hint on finding an optimal scheduling scheme than the PE utilization. In our experiments, the primary optimization strategy of NeuroSpector finds optimal scheduling solutions for 83% of the test cases.

E. Factorization and Mapping Flexibility

DNN parameters are factorized to generate the possible combinations of scheduling options when filling up the scheduling table. The supplementary strategy of NeuroSpector attempts to provide upper-level components in the accelerator hierarchy with the mapping values of large factorization degrees (e.g., 16 rather than 19) even if such scheduling options increase the energy and cycle costs of the DRAM and lower-level buffers. This optimization strategy avoids passing large prime numbers to upper-level components, thereby increasing the number of possible options to fill in the scheduling table at the later steps of the optimization flow.

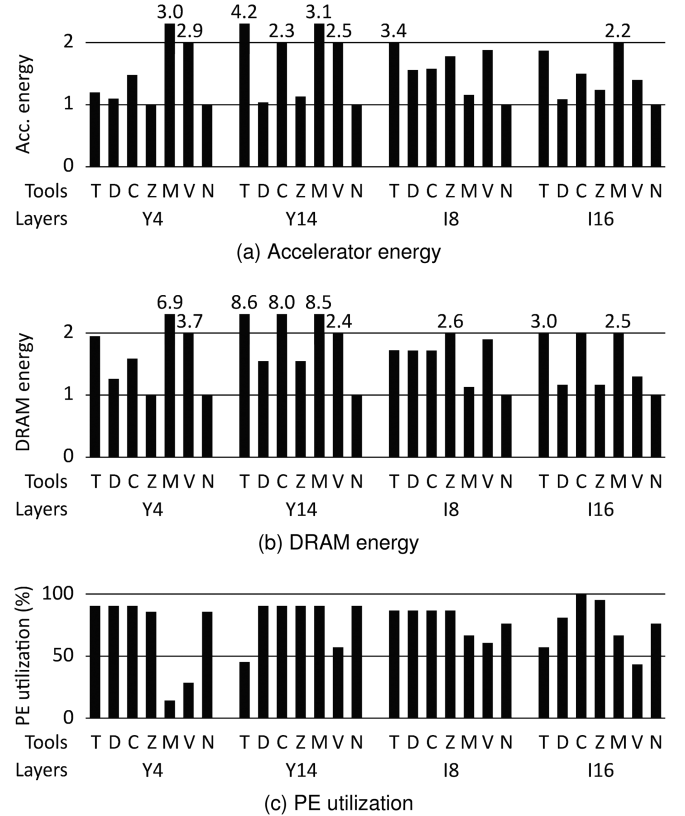


Fig. 15. (a) Total accelerator energy normalized to optimal scheduling, (b) respective DRAM energy costs, and (c) PE utilization for processing two layers of YOLO v3 (i.e., Y4, Y14) and two layers of Inception v4 (i.e., I8 and I16) on Eyeriss using the scheduling schemes derived Timeloop (T), dMazeRunner (D), CoSA (C), ZigZag (Z), Mind Mappings (M), TVM (V), and NeuroSpector (N). Reducing DRAM energy costs effectively finds energy-efficient scheduling solutions, but PE utilization ratios show less correlation with the total accelerator energy.

Fig. 16 plots such cases where enhancing the scheduling flexibility turns out to be more effective than minimizing costly accesses to the off-chip memory. Fig. 16(b) reveals that the supplementary strategy (denoted as SP) substantially increases the number of available mapping options at the local buffer level in the scheduling table, resulting in higher buffer utilization as shown in Fig. 16(c). Consequently, Fig. 16(a) shows that the improved scheduling flexibility gives lower accelerator energy than the primary optimization strategy (denoted as PM) for these neural layers. The supplementary strategy of NeuroSpector covers 17% of the test cases in our experiments.

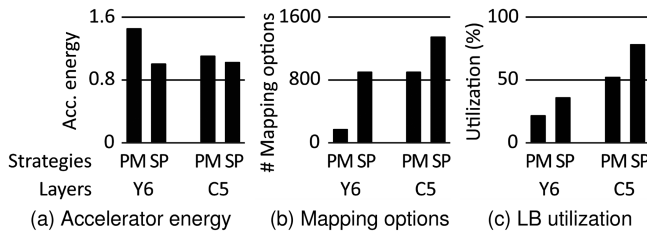


Fig. 16. (a) Total accelerator energy, (b) the number of mapping options available at the local buffer level in the scheduling table, and (c) the utilization of local buffers for executing a YOLO v3 layer (Y6) and CvT layer (C5) on Eyeriss. The supplementary strategy (SP) of NeuroSpector ensures that upper-level components are given sufficient diversity of mapping options to explore by avoiding large prime numbers. Increases in the number of possible mapping options find better scheduling solutions than the primary strategy (PM) for the shown cases.

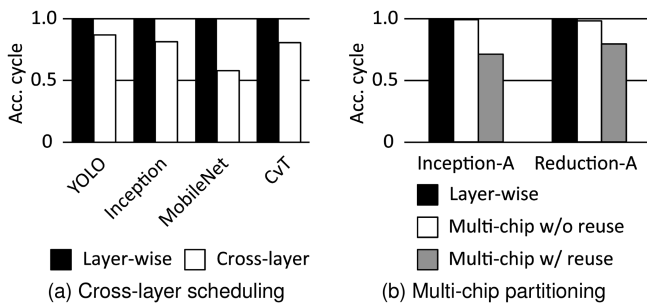


Fig. 17. (a) Cross-layer scheduling optimization of NeuroSpector achieves 23.2% reduction in execution cycles compared to per-layer scheduling. (b) Concurrent executions of independent Inception layers on multi-chip Simba save 24.7% of execution cycles, where the saving comes mostly from spatial data reuse between multi-chips.

F. Network-Level Scheduling Optimization

NeuroSpector supports cross-layer scheduling optimization by considering data reuse cases between neighboring layers in a deep neural network. Fig. 17(a) compares normalized execution cycles between per-layer and cross-layer scheduling optimization results on multi-chip Simba. The graph shows that cross-layer scheduling reduces the total execution cycles by 23.2% on average by getting rid of unnecessary data transfers between the DRAM and global buffers when proceeding from one layer to the next. Interestingly, we did not observe dramatic changes in the resulting scheduling schemes between the layer-wise and cross-layer optimizations because the per-layer optimization solutions of NeuroSpector are already near-optimal. Therefore, cross-layer scheduling does not radically alter the scheduling methods of individual layers. The results of cross-layer scheduling optimization give an insight into how much improvement can potentially be made by reusing data between successive layers in the neural networks.

Fig. 17(b) shows the effect of multi-chip partitioning for the concurrent executions of independent layers in Inception v4. The results show that multi-chip partitioning alone does not provide noticeable improvements (i.e., the case of multi-chip w/o reuse in the graph) over layer-wise executions. It implies that the per-layer scheduling optimization of NeuroSpector has

already fully utilized the hardware resources of multi-chip accelerators. The actual benefit of multi-chip partitioning comes from sharing input data between parallel layers that originate from the output of the same root layer. By eliminating redundant memory accesses to load the same input data, spatial data reuse across multi-chips saves 24.7% of execution cycles on average. The cases of cross-layer scheduling and multi-chip partitioning demonstrate the versatility and robustness of the NeuroSpector optimization methodology.

G. Consideration for Static Power

When evaluating the energy costs of scheduling options, the prior techniques [2], [7], [13], [17], [28], [29], [30], [37], [41] considered only dynamic energy expenses to account for operational activities in neural accelerators. Such an approach occasionally produces an irrational scheduling solution where only a few PEs are activated with extremely long execution cycles. For instance, Timeloop finds a scheduling option with 45.2% PE utilization for processing a YOLO layer (i.e., Y14) on Eyeriss, whereas the optimal scheduling scheme derived from NeuroSpector utilizes 90.5% of PEs for the same test. Notably, the scheduling option of Timeloop takes 8.42x longer cycles than NeuroSpector. To suppress such irrational results, dMazeRunner and ZigZag leverage utilization thresholds to preclude scheduling options that exhibit low PE utilization. However, simply relying on the PE utilization is prone to misleading scheduling options (i.e., encountering algorithmic dead ends) and still involves large problem spaces to search, as discussed in Sections V-B and V-C.

VI. CONCLUSION

The proposed methodology of NeuroSpector for scheduling optimization is demonstrated to work effectively with various neural accelerators and networks. It is proven effective, lightweight, and extensible to various optimization cases, such as multi-chip partitioning and cross-layer optimization. NeuroSpector can be used for numerous related research, encompassing scheduling optimization, design space explorations, energy and cycle cost evaluations, etc. The NeuroSpector framework is available at <http://github.com/yonseiicsl/neurospector>.

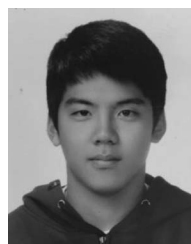
REFERENCES

- [1] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, "DORY: Automatic end-to-end deployment of real-world DNNs on low-cost IoT MCUs," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1253–1268, Aug. 2021.
- [2] P. Chatarasi, H. Kwon, A. Parashar, M. Pellauer, T. Krishna, and V. Sarkar, "Marvel: A data-centric approach for mapping deep learning operators on spatial accelerators," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 1, Dec. 2021, Art. no. 26.
- [3] T. Chen et al., "TVM: An automated end-to-end optimizing compiler for deep learning," in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2018, pp. 579–594.
- [4] Y. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture*, 2016, pp. 367–379.

- [5] Y. Chen, T. Krishna, J. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [6] Y. Chen, T. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Trans. Emerg. Sel. Topics Circuits Syst.*, vol. 9, no. 2, pp. 292–308, Jun. 2019.
- [7] S. Dave, Y. Kim, S. Avancha, K. Lee, and A. Shrivastava, "dMazeRunner: Executing perfectly nested loops on dataflow accelerators," *ACM Trans. Embedded Comput. Syst.*, vol. 18, no. 5s, pp. 1–27, Oct. 2019.
- [8] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.*, 2019, pp. 4171–4186.
- [9] A. Dosovitskiy et al., "An image is worth 16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–21.
- [10] S. Ghodrati et al., "Planaria: Dynamic architecture fission for spatial multi-tenant acceleration of deep neural networks," in *Proc. IEEE/ACM Int. Symp. Microarchitecture*, 2020, pp. 681–697.
- [11] A. Gondimalla, N. Chesnut, M. Thottethodi, and T. Vijaykumar, "SparTen: A sparse tensor accelerator for convolutional neural networks," in *Proc. IEEE/ACM Int. Symp. Microarchitecture*, 2019, pp. 151–165.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [13] K. Hegde, P. Tsai, S. Huang, V. Chandra, A. Parashar, and C. Fletcher, "Mind mappings: Enabling efficient algorithm-accelerator mapping space search," in *Proc. ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2021, pp. 943–958.
- [14] R. Hojajabr, A. Sedaghati, A. Sharifian, A. Khonsari, and A. Shriraman, "SPAGHETTI: Streaming accelerators for highly sparse GEMM on FP-GAs," in *Proc. IEEE Int. Symp. High- Perform. Comput. Architecture*, 2021, pp. 84–96.
- [15] A. Howard et al., "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 1314–1324.
- [16] A. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," Apr. 2017, *arxiv:1704.04861*.
- [17] Q. Huang et al., "CoSA: Scheduling by constrained optimization for spatial accelerators," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture*, 2021, pp. 554–566.
- [18] N. Jouppi et al., "A domain-specific supercomputer for training deep neural networks," *Commun. ACM*, vol. 63, no. 7, pp. 67–78, Jul. 2020.
- [19] N. Jouppi, C. Young, N. Patil, and D. Patterson, "Motivation for and evaluation of the first tensor processing unit," *IEEE Micro*, vol. 38, no. 3, pp. 10–19, May/Jun. 2018.
- [20] N. Jouppi et al., "In-datacenter performance analysis of a tensor processing unit," in *Proc. ACM/IEEE Int. Symp. Comput. Architecture*, 2017, pp. 1–12.
- [21] S. Kao and T. Krishna, "GAMMA: Automating the HW mapping of DNN models on accelerators via genetic algorithm," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2020, pp. 1–9.
- [22] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [23] S. Kirkpatrick, C. Gelatt, and M. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, May 1983.
- [24] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [25] H. Kwon, P. Chatarasi, M. Pellauer, A. Parashar, V. Sarkar, and T. Krishna, "Understanding reuse, performance, and hardware cost of DNN dataflow: A data-centric approach," in *Proc. IEEE/ACM Int. Symp. Microarchitecture*, 2019, pp. 754–768.
- [26] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling flexible dataflow mapping over DNN accelerators via reconfigurable interconnects," in *Proc. Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2018, pp. 461–475.
- [27] S. Li, K. Chen, J. Ahn, J. Brockman, and N. Jouppi, "CACTI-P: Architecture-level modeling for SRAM-based structures with advanced leakage reduction techniques," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2011, pp. 694–701.
- [28] L. Mei, P. Houshmand, V. Jain, S. Giraldo, and M. Verhelst, "ZigZag: Enlarging joint architecture-mapping design space exploration for DNN accelerators," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1160–1174, Aug. 2021.
- [29] G. Moon, H. Kwon, G. Jeong, P. Chatarasi, S. Rajamanickam, and T. Krishna, "Evaluating spatial accelerator architectures with tiled matrix-matrix multiplication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 4, pp. 1002–1014, Apr. 2022.
- [30] A. Parashar et al., "Timeloop: A systematic approach to DNN accelerator evaluation," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2019, pp. 304–315.
- [31] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 779–788.
- [32] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," Apr. 2018, *arxiv:1804.02767*.
- [33] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 4510–4520.
- [34] Y. Shao et al., "Simba: Scaling deep-learning inference with multi-chip module-based architecture," in *Proc. IEEE/ACM Int. Symp. Microarchitecture*, 2019, pp. 14–27.
- [35] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. Alemi, "Inception-v4, Inception-ResNet and the impact of residual connections on learning," in *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.
- [36] C. Szegedy, V. Vanhoucke, S. Ioffe, and J. Shlens, "Rethinking the inception architecture for computer vision," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2818–2826.
- [37] R. Venkatesan et al., "MAGNet: A modular accelerator generator for neural networks," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–8.
- [38] H. Wu et al., "CvT: Introducing convolutions to vision transformers," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 22–31.
- [39] Y. Wu, J. Emer, and V. Sze, "Accelergy: An architecture-level energy estimation methodology for accelerator designs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des.*, 2019, pp. 1–8.
- [40] J. Xing, L. Wang, S. Zhang, J. Chen, A. Chen, and Y. Zhu, "Bolt: Bridging the gap between auto-tuners and hardware-native performance," in *Proc. Mach. Learn. Syst.*, 2022, pp. 204–216.
- [41] X. Yang et al., "Interstellar: Using Halide's scheduling language to analyze DNN accelerators," in *Proc. IEEE Int. Conf. Architectural Support Program. Lang. Operating Syst.*, 2020, pp. 369–383.
- [42] Y. Zhao, C. Li, Y. Wang, P. Xu, Y. Zhang, and Y. Lin, "DNN-Chip predictor: An analytical performance predictor for DNN accelerators with various dataflows and hardware architectures," in *Proc. Int. Conf. Acoust. Speech Signal Process.*, 2020, pp. 1593–1597.



Chanho Park received the BS degree in electrical engineering from Kyungpook National University, Daegu, South Korea, in 2019. He is currently working toward the PhD degree with the School of Electrical and Electronic Engineering, Yonsei University, Seoul, South Korea. His research interests include design space explorations and optimization of deep neural network accelerators and machine-learning-based power modeling.



Bogil Kim received the BS degree in electrical and electronic engineering from Yonsei University, Seoul, South Korea, in 2018. He is currently working toward the PhD degree with the School of Electrical and Electronic Engineering, Yonsei University. His research interests include neural network optimization, workload characterization, and hardware accelerators. He has been the recipient of a PhD scholarship from the National Research Foundation (NRF) of Korea since 2021.



Sungmin Ryu received the BS and MS degrees in electrical and electronic engineering from Yonsei University, Seoul, South Korea, in 2020 and 2022, respectively. His research interests include machine learning techniques and software development. He is currently a quant developer with Kyobo Securities.



William J. Song (Member, IEEE) received the PhD degree in electrical and computer engineering from the Georgia Institute of Technology, Atlanta, GA, in 2015. He is an associate professor with the School of Electrical and Electronic Engineering, Yonsei University, Seoul, South Korea. His research interests include heterogeneous computing, deep learning accelerators, and power, thermal, and reliability management techniques. Before joining Yonsei University, he was a senior engineer with Intel in Santa Clara, CA. He was also a graduate intern with Qualcomm,

IBM T. J. Watson Research Center, AMD Research, and Sandia National Labs. He received the Distinguished Faculty Award from Yonsei University in 2018 and the Teaching Excellence Awards in 2018, 2020, and 2022. He was a recipient of an IBM/SRC graduate fellowship from 2012 to 2015. He received the Best Student Paper Award at IEEE International Reliability Physics Symposium in 2015.